
Modular Programming – Practical Work

Week 8

Interface, introduction GUI

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Monday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one quarter of the intermediate test note

Conventions about fonts:

- `Lucida Console` is used for Java reserved commands
- `Courier` is used for the code
- Arial is used for the text

W8Q1 – 1

1. What is happening when we forget to add a graphic component in a container (as a JFrame for example)?

.....

.....

.....

.....

.....

2. What is happening when we forget to register an event manager for a graphic component?

.....

.....

.....

.....

.....

W8Q2 – 1

The following code is based on the use of CesarCipher.java (solution of TP week 1). Analyse the code and

1. Draw manually the GUI that is generated
2. Explain:
 - a. What is going on when the user is pressing « enter » in one of the fields
 - b. What is going on when the user selects the CheckBox

See the code below :

```

import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;

import javax.swing.JCheckBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;

public class CaesarCipherGui extends JFrame {

    private JLabel inputLabel;

    private JLabel cipherLabel;

    private JTextField inputField;

    private JTextField cipherField;

    private JCheckBox cipherFlagBox;

    public CaesarCipherGui() {
        super("Cesar Cipher Demo");
        setSize(640, 125);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        setLayout(new FlowLayout()); // set frame layout

        inputLabel = new JLabel("clear text:");
        add(inputLabel);

        inputField = new JTextField("", 50);
        add(inputField);

        cipherLabel = new JLabel("ciphered :");
        add(cipherLabel);

        cipherField = new JTextField("", 50);
        cipherField.setEditable(false);
        add(cipherField);

        cipherFlagBox = new JCheckBox("Ciphering", true);
        add(cipherFlagBox);

        TextFieldHandler tHandler = new TextFieldHandler();
        inputField.addActionListener(tHandler);
        cipherField.addActionListener(tHandler);
    }
}

```

```

        CheckBoxHandler cHandler = new CheckBoxHandler();
        cipherFlagBox.addItemListener(cHandler);
    }

    private class TextFieldHandler implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            if (event.getSource() == inputField) {
                cipherField.setText(CesarCipher.cipher(
                    event.getActionCommand(), 1));
            } else if (event.getSource() == cipherField) {
                inputField.setText(CesarCipher.decipher(event
                    .getActionCommand(), 1));
            }
        } // end method actionPerformed
    }

    // private inner class for ItemListener event handling
    private class CheckBoxHandler implements ItemListener {
        public void itemStateChanged(ItemEvent event) {
            if (cipherFlagBox.isSelected()) {
                inputField.setEditable(true);
                cipherField.setEditable(false);
                inputField.requestFocus();
                inputField.selectAll();
            } else {
                inputField.setEditable(false);
                cipherField.setEditable(true);
                cipherField.requestFocus();
                cipherField.selectAll();
            }
        } // end method itemStateChanged
    } // end private inner class CheckBoxHandler

    /**
     * @param args
     */
    public static void main(String[] args) {
        CaesarCipherGui cesarGui = new CaesarCipherGui();
        cesarGui.setVisible(true);
    }
}

```

W8Q3 – 1 Œuf de Pâques : interface, héritage et enum (FR)

Implémenter les classes et interfaces suivantes :

Interfaces:

1. Une interface `Wrappable` (Emballage) avec les énumérations suivantes : couleur de l'emballage (`Colors: BLUE, RED, YELLOW, GREEN, PINK, PURPLE`) et design de l'emballage (`Pattern: CHECKED, STRIPED, DOTTED`). Les signatures des getter et des setter pour les attributs (`Colors` et `Pattern`) sont à définir.
2. Une interface `Hideable` (Caché).
3. Une interface `Colorable` (« Coloriable ») avec la signature pour la méthode `paint(Couleur color)`. L'interface contient une énumération des couleurs disponibles pour les œufs de Pâques (`enum Couleur`).

Classes:

1. `Egg` (œuf) possède un poids (`weight`). C'est une classe abstraite.
2. `ChocolateEgg` (œuf en chocolat) sont des œufs, qui peuvent être cachés et emballés. L'emballage est défini par une couleur et un design d'emballage.
3. `FilledEgg` (œuf rempli) sont des œufs en chocolat qui contiennent un type de remplissage (`FillingTyp: MARZIPAN, CREAM, ALCOHOLIC`) et un goût de remplissage (`Flavor: SCHOKOLADE, ZARTBITTER, ERDBEER, VANILLE, KROKANT`).
4. `HollowEgg` (œuf vide) sont des œufs en chocolat mais sans particularité.
5. `HensEgg` (œuf de poule) sont des œufs qui contiennent un `Status: RAW, HARDBOILED` (cru et cuit) et une couleur de coquille (`ShellColor: WHITE, BROWN`).
6. `EasterEgg` (œuf de Pâques) sont des œufs de poule qu'on peut colorier et cacher.

Pour chaque classe ajoutez un constructeur qui remplit chaque attribut par défaut et un constructeur qui passent en paramètre chaque attribut. Pour chaque classe implémentez la méthode `toString()`, qui appelle la méthode `toString()` de la superclasse et y ajoute l'affichage des attributs de la classe en cours. Si nécessaire, ajoutez d'autres méthodes ou attributs selon vos besoins.

Programmez la classe `EasterEggFactor` qui contient un `main()`.

1. Avec les méthodes de cette classe vous devez pouvoir créer un panier de Pâques qu'on remplit avec un certain nombre d'œufs. La classe contient un tableau de type `Egg`. Le nombre d'œuf dans le panier est demandé à l'utilisateur et implémenté dans le constructeur de la classe.
2. Implémentez une méthode `createSet()` qui ajoute au tableau d'œufs des œufs de différents types de façon aléatoire. Pour chaque type appelez le constructeur correspondant.
3. Implémentez la méthode `paint()`, qui sélectionne une couleur pour chaque œuf qu'il est possible de colorier. Attention seulement des œufs durs sont coloriables.
4. Implémentez une méthode `print()` qui affiche le contenu du panier. Cette méthode affiche également combien d'œufs en chocolat et combien d'œufs de Pâques (seulement les cachés) contient le panier.

W8Q3 – 1 Ostereier : Schnittstellen, Vererbung, Enum (DE)

Implementieren Sie die folgenden Klassen und Schnittstellen

Schnittstellen:

1. Eine Schnittstelle `Wrappable` (verpackbar) mit Aufzählungstypen für Verpackungsfarben (`Colors: BLUE, RED, YELLOW, GREEN, PINK, PURPLE`) und Verpackungsmuster (`Pattern: CHECKED, STRIPED, DOTTED`). Weiterhin werden Signaturen für Getter und Setter für diese Attribute (`Colors, Pattern`) definiert.
2. Eine Markierungsschnittstelle `Hideable` (versteckbar).
3. Eine Schnittstelle `Colorable` (färbbar) mit der Signatur für eine Methode `paint()`. Sie beschreibt als einzigen Parameter einen String, der die Farbe für ein gefärbtes Osterei bereitstellt. Die Schnittstelle enthält ausserdem eine Aufzählung mit den verfügbaren Farben für Ostereier.

Klassen:

7. `Egg` (Eier) haben ein Gewicht (`weight`). Die Klasse ist als abstrakte Klasse zu implementieren.
8. `ChocolateEgg` (Schokoladeneier) sind Eier, die versteckt werden können und verpackbar sind. Sie haben eine Verpackung, die durch eine Farbe und ein Muster gekennzeichnet sind.
9. `FilledEgg` (GefüllteEier) sind Schokoladeneier, die einen Füllungstyp (`FillingTyp: MARZIPAN, CREAM, ALCOHOLIC`) und Füllungsgeschmack (`Flavor: SCHOKOLADE, ZARTBITTER, ERDBEER, VANILLE, KROKANT`) haben. Die Klasse enthält jeweils eine Aufzählung für die zulässigen Werte sowie jeweils ein entsprechendes Attribut.
10. `HollowEgg` (Hohleier) sind ebenfalls Schokoladeneier, haben aber keine Besonderheiten.
11. `HensEgg` (Hühnereier) sind Eier, die durch einen Status: `RAW, HARDBOILED` (roh, hartgekocht) und die Farbe der Schale (`ShellColor: WHITE, BROWN`) gekennzeichnet sind. Verwalten Sie diese Werte ebenfalls über Aufzählungstypen.
12. `EasterEgg` (Ostereier) sind Hühnereier, die färbbar und versteckbar sind.

Erstellen Sie für jede Klasse jeweils einen Konstruktor, der alle Attribute per Zufall bestimmt (bzw. das Gewicht konstant festlegt) und einen weiteren, dem alle benötigten Werte als Parameter übergeben werden können. Weiterhin programmieren Sie für alle Klassen die Methode `toString()`, die jeweils dieselbe Methode der Oberklasse aufruft und im Anschluss die eigenen Attribute ergänzt. Weitere Methoden und Attribute fügen Sie nach Bedarf hinzu.

Erstellen Sie eine Klasse `EasterEggFactory`. In dieser Klasse kann auch das `main()` enthalten sein.

5. Mit den Methoden dieser Klasse sollen Osterkörbe mit einer von Anwender zu bestimmenden Anzahl Eiern gefüllt werden. Die Klasse verwaltet ein Array vom Typ `Egg`. Über den Konstruktor kann bestimmt werden, wie viele Ei-Objekte im Array verwaltet werden. Fragen Sie den Anwender im `main()`, wie viele Eier sein Korb umfassen soll.

6. Implementieren Sie eine Methode `createSet()`. Welche Eier in den Korb gelegt werden, wird jeweils vom Zufall bestimmt (verwenden Sie die hierfür bereitgestellten Konstruktoren). Verwalten Sie auch hier einen Aufzählungstyp mit den verfügbaren Eiarten, so dass Sie per Zufall auswählen können, welcher Typ Ei erstellt wird.
7. Implementieren Sie eine Methode `paint()`, die für alle färbbaren Eier eine zufällige Farbe auswählt und setzt. Berücksichtigen Sie, dass nur hartgekochte Eier gefärbt werden können.
8. Programmieren Sie eine Methode `print()`, die den Inhalt des Korbes ausgibt. Zählen Sie, wie viele Schokoladeneier und wie viele Ostereier im Korb liegen (nur versteckbare Eier sind zu zählen!)